Day 30 – Professional Environment Integration

Mission

Demonstrate that you can configure, use, and validate a complete developer environment as a professional software engineer.

By the end of today you should be able to:

- create and organize a CLI project
- use a Python virtual environment
- manage dependencies
- execute the application
- validate functionality
- use Git professionally
- produce engineering documentation
- archive evidence exactly as required

This day is essentially your **Week 6 practical examination**.

Engineering Objectives

Today's objectives are to demonstrate:

- Environment competency
- Python environment competency
- Git workflow competency
- Repository organization
- Documentation discipline
- Repeatable engineering workflow

Exercises

Exercise 1 — Environment Verification

Verify your complete workstation.

Check:

- Python version
- pip version
- virtual environment activation
- Git version
- current repository
- current branch
- working tree status

Expected outcome:

Everything reports correctly and the repository is clean.

Exercise 2 — Project Bootstrap

Create a new Week 6 project.

Suggested project:

```
developer_environment_demo
```

Project structure:

```
developer_environment_demo/
│
├── main.py
├── requirements.txt
├── README.md
├── .gitignore
└── venv/
```

Your goal is to create this quickly using the workflow you've developed throughout the week.

Exercise 3 — Virtual Environment

Create and activate a virtual environment.

Validate:

- activation
- interpreter path
- installed packages

- deactivate
- reactivate

Expected result:

You can repeatedly activate and deactivate without errors.

Exercise 4 — Dependency Management

Install at least one external package.

Good candidates include:

- colorama
- rich

Create or update:

```
requirements.txt
```

Then verify that the project can be recreated using only:

```
pip install -r requirements.txt
```

This reinforces the practice you established on Day 28.

Exercise 5 — Git Workflow

Perform a complete professional Git cycle.

Verify:

```
git status

git add

git status

git commit

git log

git push
```

```
git fetch

git status
```

Document every major state transition.

Mini Project

Professional Developer Workspace Demonstration

Create a small command-line application that proves your development environment is functioning correctly.

Suggested behavior

When executed:

```
Developer Environment Demonstration

=================================

Python Version:
...

Virtual Environment:
...

Current Working Directory:
...

Git Repository:
...

Installed Packages:
...

Current Time:
...

Status:
Developer environment successfully verified.
```

Functional Requirements

The application should demonstrate:

Environment

- Python version
- executable path
- platform
- current directory

Virtual Environment

Detect whether the application is executing inside a virtual environment.

Package Verification

Successfully import the package(s) installed in your virtual environment.

Filesystem

Display:

- current directory
- project files

Git

Optionally execute read-only Git commands such as:

```
git status --short
```

or

```
git branch
```

using `subprocess`.

Avoid making repository changes from the program itself.

Output

Produce a clean, professional console report.

Stretch Goals

If time permits, consider adding:

- ANSI colors (Colorama or Rich)
- execution timer
- formatted section headers
- configuration file support
- command-line options using `argparse`

These are enhancements, not core requirements.

Validation Matrix

By completion, you should be able to answer "Yes" to each item:

Validation	Complete
Python environment works	☐
Virtual environment functions correctly	☐
Dependencies install from `requirements.txt`	☐
CLI application executes successfully	☐
Git workflow completed	☐
Evidence captured	☐
Notes.md completed	☐
Journal.md completed	☐
Changes committed and pushed	☐

Evidence Checklist

Capture evidence for:

- Project directory structure
- Virtual environment activation
- Package installation
- Successful application execution
- Git status before commit
- Git commit
- Git log
- Git push
- Final clean repository status

Engineering Review

Today's emphasis is not on algorithmic complexity but on demonstrating disciplined engineering practice. A professional developer should be able to provision a project, isolate dependencies, verify the environment, manage source control, and leave the repository in a clean, reproducible state.

Week 6 Completion

After Day 30, Week 6 should demonstrate that you have achieved the following success criteria:

- ✓ Environment competency
- ✓ Engineering workflow competency
- ✓ Complete documentation
- ✓ Git repository synchronized
- ✓ Evidence captured

This closes **Phase II – Professional Engineering Practice** and positions you to begin **Week 7**, where the focus shifts from tooling and workflow to **Object-Oriented Programming and building larger software systems**.